MTM4681 · MILESTONE 1 · ARCHITECTURE & PROTOCOLS

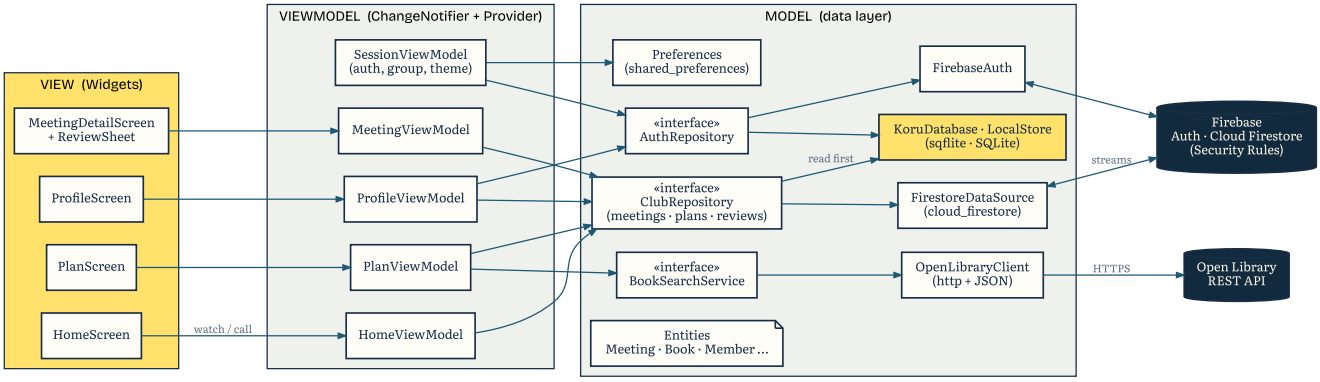
Koru: Flutter app

Team 4 · Ahmet Efe Gürel, Muhammet Kerem Aydın
Repo: github.com/muhammetkeremaydin/koru-mobile-app
Figma: figma.com/design/XKC6fYGfZ3pcLfCgGDakab
Flutter 3.x · Dart 3 · Firebase · Android + iOS

1. Context

Koru is a mobile app for small, in-person book clubs: members see the next meeting, reply “Geliyorum”, review the book after the meeting and vote on the next one, while the club's history is kept automatically. The client is written in **Flutter**; cloud data lives in **Firebase** (Authentication + Cloud Firestore), which gives us accounts, real-time sync and an offline cache without writing our own server. Book metadata comes from the public **Open Library** API. On the phone, a local **SQLite** database (`sqflite`) keeps the accounts used on this device, each member's favorites, the meeting history and review drafts, so the app opens instantly and works offline. The MVP runs entirely on Firebase's no-cost Spark plan. Out of scope for the MVP: photo albums and avatar upload (Cloud Storage now requires the pay-as-you-go Blaze plan), push notifications (FCM + Cloud Functions), in-app chat and video.

2. Design pattern: MVVM + Repository (Provider)



Dependencies point one way: View → ViewModel → Repository interface → data source → Firebase / Open Library. No widget talks to Firestore directly.

Layer	Responsibility	In Koru
View (StatelessWidget / StatefulWidget)	Renders state, forwards user intent. `build()` stays pure: no network, no business rules. Uses `context.watch<VM>()` / `Consumer` .	9 screens (01–09) + reusable widgets: `KoruAppBar` , `Eyebrow` , `OutlinedCommand` , `LedgerRow` , `BookCover` , `RatingGrid` .
ViewModel (`ChangeNotifier`)	Holds screen state (`ViewState` : loading · ready · empty · error · offline), subscribes to repository streams, validates input, calls repository methods, then `notifyListeners()` .	`SessionViewModel` (sign-in, current group, theme), `HomeViewModel` , `MeetingViewModel` (RSVP, review rules), `PlanViewModel` (voting, suggestions), `ProfileViewModel` .
Model (entities + repositories)	Immutable entities with `fromFirestore` / `toFirestore` / `copyWith` ; abstract repositories define *what* operations exist; implementations decide *how* (Firestore, fake data for tests).	`Account, Group, Book, Meeting, Rsvp, Attendance, Review, Plan, Vote` ; `AuthRepository` , `ClubRepository` , `BookSearchService` , `LocalStore` (SQLite DAOs).

Why MVVM instead of MVC. Flutter is declarative: the UI is a function of state, so a ViewModel that exposes state and notifies listeners fits better than a Controller that pushes changes into views. It is also the course's target architecture (Provider in Week 11). ViewModels contain no widgets, so rules such as “rating is required when status is *read* or *partial*”, “a member has one active vote” and “RSVP closes at the end of meeting day” are unit-testable with a fake repository. **State management: Provider** (`MultiProvider` at the root, `ChangeNotifierProvider` per feature). It is simple, official and enough for about six ViewModels; Riverpod was considered but adds concepts we do not need at this scope.

Folder structure

```
lib/
├─ main.dart, app.dart        Firebase.initializeApp · MaterialApp.router · MultiProvider
├─ core/      theme/ (KoruTheme, ThemeExtension) · router/ · utils/ (TR dates, TR search normalize)
├─ data/      models/ · repositories/ (abstract) · sources/ (firestore_*, firebase_auth_*, open_library_*,
fake_*)
│            local/ (koru_database.dart, migrations.dart, account_dao, meeting_dao, review_dao)
└─ features/  auth · home · meetings · review · plan · books · profile
             └─ each: view/ · view_model/ · widgets/
test/        view_models/ · models/   (flutter_test + fake repositories)
firestore.rules                       server-side permissions (members, admins)
```

3. Screen map → ViewModel → data

#	Screen / route	ViewModel	Repository call → Firebase / API
01	Giriş `/login`	SessionVM	`signIn()` → Firebase Auth e-mail/password; saved accounts on this device from `accounts` table
02	Grup seçimi `/groups`	SessionVM	`myGroups()` → query `groups` where member; choice saved in preferences
03	Ana sayfa `/home`	HomeVM	`watchMeetings(groupId)` → snapshot stream of `meetings`
04	Buluşmalar `/meetings`	HomeVM (shared stream)	meeting history read from local `meetings` table (works offline); SQL search and filters
05	Buluşma detayı `/meetings/:id`	MeetingVM	`setRsvp()` → set/delete `rsvps/{uid}`; `toggleFavorite()`; share / calendar / maps via OS
06	Değerlendirme (sheet)	MeetingVM	`saveReview()` → set `reviews/{uid}` (validated)
07	Plan `/plan`	PlanVM	`vote(planId)` → set `votes/{uid}`; admin `scheduleMeeting(planId)` → batch write
08	Kitap öner `/plan/new`	PlanVM	`searchBooks()` → Open Library `search.json`; `addPlan()` → add to `plans`
09	Profil `/profile`	ProfileVM, SessionVM	stats and favorites via SQL on the local DB; edit name / birth year; theme in preferences; `signOut()`

Navigation: `go_router` with a global `redirect` guard driven by `FirebaseAuth.authStateChanges()` (no user → `/login?from=…`, resumed after sign-in) and a `StatefulShellRoute.indexedStack` for the four tabs, so each tab keeps its own stack. Detail screens are pushed inside their tab; the review form is a modal bottom sheet that returns its result with `context.pop(result)`.

4. Data model

Cloud: Cloud Firestore (source of truth, shared by the group)

```
users/{uid}                       name, email, birthYear?, color, favorites[bookId]
groups/{groupId}                  name, voteVisibility (open|secret), createdBy
  members/{uid}                   role (member|admin|owner), joinedAt
  books/{bookId}                  title, author, pages?, isbn?, coverUrl?   (Open Library or manual)
  meetings/{meetingId}            bookId, date, location, mapUrl?, note?, chapterRange?, status
    rsvps/{uid}                   createdAt                 ← one doc per member = no double count
    reviews/{uid}                 readingStatus, currentPage?, rating 1–10, comment?
    attendance/{uid}              present, recordedBy       (admin)
  plans/{planId}                  bookId, suggestedBy, targetDate?
  votes/{uid}                     planId                    ← one doc per member = one active vote
```

Using the member's `uid` as the document ID for RSVPs, reviews and votes enforces “one per member” by construction. **Security Rules** check that a user writes only their own `rsvps/reviews/votes` documents, that only `admin/owner` members create meetings or record attendance, and that ratings are 1–10 and pages ≥ 0. The client hides admin buttons, but the rules are the real permission check.

On the device: SQLite via sqflite (local DB, offline-first)

```
accounts    (uid TEXT PK, email TEXT, display_name TEXT, birth_year INTEGER NULL, color TEXT,
             last_login_at INTEGER, is_current INTEGER)
groups      (id TEXT PK, name TEXT)
group_memberships
            (account_uid → accounts, group_id → groups, role TEXT, PK(account_uid, group_id))
books       (id TEXT PK, title TEXT, author TEXT, pages INTEGER, isbn TEXT, cover_url TEXT)
favorites   (account_uid → accounts, book_id → books, added_at INTEGER, PK(account_uid, book_id))
meetings    (id TEXT PK, group_id → groups, book_id → books, date INTEGER, location TEXT,
             chapter_range TEXT, status TEXT, synced_at INTEGER)
my_rsvps    (account_uid, meeting_id → meetings, coming INTEGER, updated_at INTEGER,
             PK(account_uid, meeting_id))
my_reviews  (account_uid, meeting_id → meetings, reading_status TEXT, current_page INTEGER, rating INTEGER,
             comment TEXT, is_draft INTEGER, updated_at INTEGER, PK(account_uid, meeting_id))
sync_state  (collection TEXT PK, last_synced_at INTEGER)
```

Firestore stays the source of truth; the local DB is an offline-first mirror plus the member's personal data. Repositories read from SQLite first, so lists and profile stats appear immediately, and Firestore listeners upsert changes inside a transaction. A review written offline stays as a draft (`is_draft = 1`) until it is sent. Birth year is optional and only shown on the profile; passwords are never stored locally, because Firebase Auth keeps the session. “Remove account from this device” deletes that `uid` 's rows through `ON DELETE CASCADE` , then drops cached groups and books no account on the device still needs. A group is cached once and linked to each account through `group_memberships` , so two members can share one phone. The schema is versioned with `onCreate` / `onUpgrade` migrations.

5. Protocols and interfaces

Protocol / interface	Package	Purpose in the app
Firebase Authentication (HTTPS, ID tokens)	`firebase_auth`	E-mail/password sign-in, password reset, persistent session. The SDK refreshes short-lived ID tokens automatically; `authStateChanges()` drives the router guard. No secrets in the app bundle.
Cloud Firestore (gRPC/HTTP 2 over TLS, real-time listeners)	`cloud_firestore`	All club data. `snapshots()` streams feed ViewModels, so another member's RSVP or vote appears live. Writes use transactions/batches where several documents change (move plan → meeting).
Local relational DB (SQL)	`sqflite` , `path` , `shared_preferences`	SQLite tables for accounts, favorites, meeting history and review drafts (section 4); queried with SQL for search, filters and profile statistics. When offline, the app shows local data with a “çevrimdışı” badge and sends drafts on reconnect. Preferences hold only theme mode and the selected group.
REST + JSON (Open Library)	`http` , `dart:convert`	`GET https://openlibrary.org/search.json?q=…` to prefill title, author, pages and cover; 3-character minimum, 10 s timeout, manual entry as fallback for Turkish titles the catalogue misses.
Deep links	`go_router` , `app_links`	A shared meeting link opens `/meetings/:id` in the right group; after sign-in the user returns to it.
OS intents	`share_plus` , `url_launcher`	Share a meeting, add it to the calendar, open the location in maps.

Key Dart interfaces (abstract classes)

```
abstract class AuthRepository {
  Stream<AppUser?> authStateChanges();
  Future<void> signIn(String email, String password);
  Future<void> resetPassword(String email);
  Future<void> signOut();
}
abstract class ClubRepository {
  Stream<List<Meeting>> watchMeetings(String groupId);
  Stream<List<Plan>> watchPlans(String groupId);
  Future<void> setRsvp(String groupId, String meetingId, bool coming);
  Future<void> saveReview(String groupId, String meetingId, ReviewInput input);
  Future<void> vote(String groupId, String? planId);          // null = withdraw
  Future<void> addPlan(String groupId, BookDraft book, {DateTime? target});
  Future<String> scheduleMeeting(String groupId, MeetingInput input);   // admin
}
abstract class BookSearchService { Future<List<BookDraft>> search(String query); }
abstract interface class LocalStore {                         // SqliteLocalStore (sqflite)
  Future<List<Account>> accountsOnDevice();
  Future<void> rememberLogin(Account account);                // one current account
  Future<void> upsertMeetings(List<Meeting> meetings);
  Future<List<MeetingHistoryItem>> meetingHistory(String uid, String groupId, {String? query});
  Future<ProfileStats> stats(String uid);                     // attended, books, average rating
  Future<bool> saveDraft(String uid, ReviewInput draft);
  Future<void> removeAccount(String uid);                     // cascade + prune orphans
}
```

Each interface has a **Firebase** implementation and a **Fake** implementation with in-memory sample data, used in unit/widget tests and for UI work before the Firebase project is ready. Providers inject the implementation, so ViewModels never change.

6. Plan and risks

Week	Ahmet Efe Gürel	Muhammet Kerem Aydın
7 (MS1)	User flow, architecture, report	Wireframes, mockups, prototype
8	Firebase project, `flutterfire configure`, models, Security Rules draft	`KoruTheme`, shared widgets, router + tab shell
9	Auth + group selection, local SQLite DB (schema, DAOs, sync)	Home + meetings list (search/filter)
10	Meeting detail: RSVP, favorite, review sheet	Plan: voting, book search, add plan
11	Provider wiring, ViewModel and DB migration tests, rules tests	Profile, statistics, theme switch
12–13	Deep links, share/calendar, error states	Animations, accessibility pass, dark mode QA
14	Real-device test (Android + iOS), APK, README screenshots, demo video	

- **Security Rules mistakes** could expose or block data; we mitigate this by writing rules with the data model and testing them in the Firebase Emulator Suite.
- **Free-tier limits:** Spark has daily read/write quotas; streams are scoped to one group and lists are paginated. Photo features would need Blaze (Cloud Storage) and are left for later.
- **Local and cloud out of sync:** Firestore always wins, except unsent drafts; offline scenarios are part of the test plan.
- **Turkish book metadata:** Open Library misses many Turkish editions; manual entry is a first-class path, not an error state.
- **iOS builds need macOS/Xcode:** Android is the primary test target; iOS verified on a Mac or via CI.